

Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

 /c/Users/acom/Desktop/OS_CD24037

```
acom@DESKTOP-50T9G9Q MSYS /c/Users/acom/Desktop/OS_CD24037
$ nano rohit1.c

acom@DESKTOP-50T9G9Q MSYS /c/Users/acom/Desktop/OS_CD24037
$ gcc rohit1.c -o rohit1

acom@DESKTOP-50T9G9Q MSYS /c/Users/acom/Desktop/OS_CD24037
$ ./rohit1
System is in SAFE state.
Safe Execution Order:
Firefox -> Excel -> PowerPoint -> Google Drive -> Word Processor

acom@DESKTOP-50T9G9Q MSYS /c/Users/acom/Desktop/OS_CD24037
$ |
```

/c/Users/acom/Desktop/OS_CD24037

```
GNU nano 8.7
#include <stdio.h>

int main() {
    int n = 5; // Number of processes
    int m = 4; // Number of resource types

    // Process Names
    char process[5][20] = {
        "Google Drive",
        "Firefox",
        "Word Processor",
        "Excel",
        "PowerPoint"
    };

    // Allocation Matrix
    int allocation[5][4] = {
        {0, 1, 0, 1},
        {2, 0, 0, 1},
        {3, 0, 2, 1},
        {2, 1, 1, 0},
        {0, 0, 2, 1}
    };

    // Max Matrix
    int max[5][4] = {
        {7, 5, 3, 4},
        {3, 2, 2, 2},
        {9, 0, 2, 2},
        {2, 2, 2, 2},
        {4, 3, 3, 3}
    };

    // Available Resources
    int available[4] = {3, 3, 2, 2};

    int need[5][4];
    int finish[5] = {0};
    int safeSeq[5];
    int work[4];

    // Calculate Need Matrix
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Copy available to work
    for(int i = 0; i < m; i++)
        work[i] = available[i];

    int count = 0;
```

AG Help AO Write Out AF Where Is AK Cut
AX Exit AR Read File \ Replace AU Pas

/c/Users/acom/Desktop/OS_CD24037

```
GNU nano 8.7
        for(int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Copy available to work
    for(int i = 0; i < m; i++)
        work[i] = available[i];

    int count = 0;

    while(count < n) {
        int found = 0;
        for(int i = 0; i < n; i++) {
            if(finish[i] == 0) {
                int possible = 1;
                for(int j = 0; j < m; j++) {
                    if(need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if(possible) {
                    for(int j = 0; j < m; j++)
                        work[j] += allocation[i][j];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("System is NOT in safe state.\n");
            return 0;
        }

        printf("System is in SAFE state.\n");
        printf("Safe Execution Order:\n");

        for(int i = 0; i < n; i++) {
            printf("%s", process[safeSeq[i]]);
            if(i != n-1)
                printf(" -> ");
        }

        printf("\n");

        return 0;
    }
}
```

AG Help AO Write Out AF Where Is AK Cut
AX Exit AR Read File \ Replace AU Paste